

BLG 483E — Quiz 4 Part I

salihsefer36

2026-05-10

Contents

Part 1: Academic Background & Course Preferences	3
Perceived Strengths (Top 3 Easy Courses)	3
Perceived Challenges (Top 3 Difficult Courses)	3
Selected Sub-topic: Pipelining Hazards	3
Why this was difficult for me	3
Prompt Engineering Log — Pipelining Hazards	4
1. Initial Prompt	4
2. Design Choice	4
3. Iterative Refinement Process	4
v1 → v2 (Iteration 1)	4
v2 → v3 (Iteration 2)	5
v3 → v4_final (Iteration 3)	5
4. Before vs After	6
Part 3: Comparative Performance Analysis — Pipelining Hazards vs. Instructor Slides	8
1. Communication Strategy	8
2. Information Density (TikZ Prompt)	8
3. Efficiency	9
4. Slide Improvement Challenge	10
5. Missing Piece	11
Part 4: Quality Assurance and Interactive Testing	12
a) Verification Quiz	12
Question 1 — Visual / TikZ Interpretation	12
Question 2 — Logical Derivation / Calculation	13
b) Decompression Task	14
Part 5: Feedback on AI Synthesis	18
1. Hardest Elements to Get the AI to Generate Correctly	18
TikZ Diagrams	18
Mathematical Notation	18
Pedagogical Balance	18
2. Repeated AI Mistakes	18

3. Golden Rule Prompt 19

Field	Value
Course	BLG 483E Artificial Intelligence
CRNs	25311 / 25410
Student	salihsefer36
Topic	Pipelining Hazards in CPU Architecture
Date	2026-05-10
Iterations	v1 → v2 → v3 → v4_final
Final PDF	Final_Slides.pdf

Part 1: Academic Background & Course Preferences

Perceived Strengths (Top 3 Easy Courses)

Course Name	Grade	Reason
Numerical Methods	BA	I could visualize the numerical methods
Linear Algebra	BA	Step-by-step matrix applications
OOP	BB	Clear real-world analogies

Perceived Challenges (Top 3 Difficult Courses)

Course Name	Grade	Reason
Digital Circuits	CB	Digital Circuit analogies
Computer Organization	CC	First Assembly experience
Computer Architecture	CB	Pipelining/caching hard to visualize

Selected Sub-topic: Pipelining Hazards

Why this was difficult for me

Pipelining Hazards was one of the most challenging topics I encountered in Computer Architecture because the problem is fundamentally invisible — unlike a wrong formula result, a data hazard silently corrupts execution, making it hard to develop intuition without careful cycle-by-cycle tracing. The five-stage pipeline (IF-ID-EX-MEM-WB) made sense in isolation, but reasoning about multiple instructions overlapping in different stages simultaneously stretched my mental model well beyond what I was used to in sequential programming. Structural hazards, data hazards (RAW, WAW, WAR), and control hazards each required a different mitigation strategy — stalls, forwarding, or branch prediction — and keeping all three straight while also calculating stall counts felt like juggling too many moving parts at once. What finally helped was drawing out the pipeline as a time-step grid on paper, which is exactly the kind of visual scaffold I needed but that the course lacked.

Prompt Engineering Log — Pipelining Hazards

1. Initial Prompt

The following condensed command was used to generate `v1.tex`:

```
Create quiz4-part1/slides/v1.tex. Topic: Pipelining Hazards in CPU Architecture.
Build deliberately weak - plain text only, no TikZ, no real-world analogy, no worked
example. Structure: 6 slides (title + 5 content slides + 1 references slide).
Requirements: \documentclass{beamer}, \usetheme{Madrid},
\usepackage{amsmath, amssymb, listings, xcolor}, \title{Understanding Pipelining Hazards},
frames using \begin{frame}{...}\end{frame}.
```

The five content slides covered: Introduction to CPU Pipelining, Data Hazards, Structural Hazards, Control Hazards, and Hazard Mitigation Summary.

2. Design Choice

Pipelining Hazards was selected as the topic because it was identified in the student’s course history as the most difficult sub-topic within Computer Architecture — a subject graded CB precisely because “pipelining/caching hard to visualize.” This makes it an ideal candidate for AI-assisted visual explanation. Beamer was chosen because it is the de-facto standard for academic technical presentations in computer science: it supports LaTeX math notation natively, integrates TikZ without friction, and produces professional-quality PDFs suitable for Ninova upload. The Madrid theme was used for the first three iterations because it is pre-installed in all standard TeX Live distributions, avoiding any dependency risk during iterative development. The content structure follows a deliberate **text** → **analogy** → **visual** → **polish** progression across four versions, mirroring the pedagogical arc recommended in Patterson & Hennessy’s *Computer Organization and Design* (the course textbook): establish vocabulary first, then build intuition, then provide a concrete model, then consolidate. Five content areas were chosen to match the natural decomposition of the topic: pipeline basics, data hazards (RAW/WAR/WAW), structural hazards, control hazards, and mitigation strategies.

3. Iterative Refinement Process

Four iterations were performed: `v1.tex` → `v2.tex` → `v3.tex` → `v4_final.tex`.

v1 → v2 (Iteration 1)

Strategy: “Explain the core concepts as if talking to a 10-year-old.”

Changes made: - Prepend a one-sentence real-world analogy before every technical concept on every slide (restaurant/assembly line for pipelining; relay race for RAW; pan-cleaning for WAR; report filing for WAW; loading dock for structural hazard; GPS wrong-lane for control hazard;

factory supervisor for mitigation summary). - Simplified jargon throughout by adding plain-English equivalents in parentheses (e.g., “stall” → “like holding a car at a red light for one cycle”). - Split the densest slide — Data Hazards, which had 6 bullets — into two frames: “Data Hazards — Types” and “Data Hazards — Detection & Mitigation”. - Changed `\medskip` to `\smallskip` in several places to accommodate the added analogy text without overflowing slides. - File size grew from 5 370 bytes (115 lines) to 8 019 bytes (170 lines); slide count increased from **7 to 8**.

Weakness addressed: Dense technical jargon with no scaffolding made `v1.tex` inaccessible to anyone without prior pipeline knowledge. Bullet points like “RAW (true dependence): a consumer instruction attempts to read a register operand before the producer instruction completes its WB stage” assumed fluency in micro-architecture terminology. The analogies in `v2` provide concrete mental hooks — the relay-race baton for RAW, the loading-dock for structural conflicts, the GPS recalculation for branches — that allow a reader to reason about the concepts before engaging the formal definitions.

v2 → v3 (Iteration 2)

Strategy: “Add at least 2 TikZ diagrams and one concrete numerical worked example.”

Changes made: - Added `\usepackage{tikz}` and `\usetikzlibrary{positioning, arrows.meta, shapes}` to the preamble. - Defined a global `\tikzset{}` block with six reusable styles: `stage` (large blue pipeline-stage box), `blk` (normal timing-grid cell, blue), `stl` (stall cell, red fill), `emp` (invisible spacer), `harr` (hardware arrow), `fwdarr` (green dashed forwarding arrow). - **TikZ Diagram 1 — “The 5-Stage Pipeline” (new slide 3):** horizontal block diagram with five stage-styled nodes (IF → ID → EX → MEM → WB) connected by `{Stealth}` arrows, with grey label text below each node. - **TikZ Diagram 2 — “Worked Example: RAW Hazard & Forwarding” (new slide 6):** two side-by-side timing grids: an 8-cycle grid (I1 + I2 with 2 red `stl` cells at cycles C4–C5) and a 6-cycle grid (I1 + I2 with a green dashed `fwdarr` from `i1ex.south` to `i2ex.north`); plus a `\begin{tabular}` step-by-step table comparing both cases across 8 columns. - The tabular conclusion: **without forwarding → 2 stalls (8 cycles); with forwarding → 0 stalls (6 cycles) → 25% speedup**. - PDF size grew from 103 KB to **136 KB** (embedded TikZ vector graphics). Slide count increased from **8 to 10**.

Pedagogical gap closed: `v2.tex` explained hazards through analogies and words alone but provided no visual model of concurrent stage execution. A student could understand that a stall “freezes the front of the pipeline” without being able to see *which cycle that freeze occurs in* or *how many cycles are wasted*. The TikZ timing grids make the cycle-by-cycle progression concrete and countable: the reader can point to the two red `--` cells and immediately see the 2-cycle cost. The worked example tabular provides the numerical precision needed to verify one’s own calculations.

v3 → v4_final (Iteration 3)

Strategy: “Final pedagogical polish — add learning objectives, navigation, intuition blocks, real-world context, edge cases, and a self-test.”

Changes made: - Switched Beamer theme from **Madrid** to **Metropolis** (`\usetheme{metropolis}`, `\metroset{sectionpage=none, subsectionpage=none, progressbar=frametitle}`), giving

a cleaner sans-serif layout with a thin per-slide progress bar. - Added an author note via `\institute:` “Generated with AI assistance, refined over 4 iterations.” - **New slide 2 — “Learning Objectives”:** three `\enumerate` goals using Bloom’s taxonomy verbs: *Identify* (hazard classes + analogies), *Apply* (forwarding/stall calculation), *Compare* (mitigation strategies). - **New slide 3 — “Outline”:** `\tableofcontents` populated by five `\section{}` commands — *What is Pipelining?*, *Data Hazards*, *Structural & Control Hazards*, *Mitigation Techniques*, *Summary* — compiled twice for cross-reference resolution. - Added **6** `\begin{block}{Intuition}...\end{block}` blocks at the end of all six theoretical slides (Introduction, Data Types, Data Detection, Structural, Control, Mitigation Summary), each distilling the core insight in 1–2 sentences. - **New slide 13 — “Real-World Context”:** three bullets connecting theory to practice — modern x86 (Tomahawk/register renaming, >98% branch prediction accuracy), ARM Cortex-A53 (in-order) vs. A76 (out-of-order), and GPU SIMT warp-switching as an alternative latency-hiding strategy. - **New slide 14 — “Summary & Common Pitfalls” (final):** 3 summary bullets, 2 misconceptions (“*forwarding solves all data hazards*” and “*deeper pipelines are always faster*” — with the Pentium 4 Prescott 31-stage counter-example), and a `\begin{alertblock}{Test Yourself}` presenting an unresolved load-use puzzle across three instructions. - PDF size grew from 136 KB to **153 KB**. Slide count increased from **10 to 14**.

Final polish added: The deck now has a complete pedagogical arc. Learning Objectives frame the session upfront; the ToC provides navigational structure; Intuition blocks give the “why it matters” insight after each formal definition; Real-World Context answers the “so what?” question; and the Summary/Pitfalls slide consolidates key facts while actively challenging the reader. The Metropolis theme gives the final version a visual identity distinct from the draft versions.

4. Before vs After

Initial output (v1.tex) weaknesses: - **No visual representation:** pipeline stages, timing diagrams, and hazard effects were described entirely in text. The concurrent execution of multiple instructions across stages — the defining property of pipelining — was impossible to grasp without a diagram. - **Dense technical jargon with no scaffolding:** terms like “IPC 1”, “EX/MEM and MEM/WB pipeline registers”, “BTFN (backward-taken, forward-not-taken)” appeared without explanation or analogy, making the slides inaccessible to students encountering these concepts for the first time. - **No pedagogical architecture:** no learning objectives, no outline, no summary, no self-test. The slides were a flat enumeration of facts with no structure to guide what a student should take away, check, or remember.

Final version (v4_final.tex) strengths: - **Full visual layer:** two TikZ diagrams (the IF→ID→EX→MEM→WB pipeline block diagram and the before/after timing grids with a green forwarding arrow) make hazard mechanics immediately visible; the `\begin{tabular}` table provides numerical verification. - **Three-layer pedagogical architecture:** Learning Objectives → structured content with Intuition blocks → Summary & Common Pitfalls creates a complete lesson with a clear entry, body, and consolidation phase. - **Industrial grounding:** the Real-World Context slide connects textbook theory to modern x86 out-of-order execution, ARM Cortex micro-architectures, and GPU SIMT warp-switching — answering “why does this matter?” with concrete industrial examples.

Comparison table:

Dimension	v1	v4_final
Slide count	7	14
TikZ diagrams	0	2
Worked example	none	RAW hazard timing grids + <code>\begin{tabular}</code> table
Analogy	none	6 (restaurant, relay race, traffic officer, loading dock, GPS, factory supervisor)
Learning Objectives	none	yes (<code>\begin{enumerate}</code>), 3 Bloom's-verb goals)
Intuition blocks	0	6 (<code>\begin{block}{Intuition}</code>)
Sections / ToC	none	5 <code>\section{}</code> commands + <code>\tableofcontents</code>
Beamer theme	Madrid	Metropolis (<code>progressbar=frametitle</code>)
Real-World Context	none	x86, ARM Cortex-A53/A76, GPU SIMT
Common pitfalls	none	2 misconceptions + Pentium 4 counter-example
Self-test	none	<code>\begin{alertblock}{Test Yourself}</code>
PDF size	99 KB	153 KB

Part 3: Comparative Performance Analysis — Pipelining Hazards vs. Instructor Slides

1. Communication Strategy

Among all the instructor’s sample slides, `c-real-life.pdf` (**Convolution — detective and flashlight analogy**) was the most effective communication strategy. Its central pedagogical strength is the use of a **single, sustained metaphor** that is introduced at the very beginning and then mapped element-by-element onto the formal mathematical operation: the detective (signal), the flashlight beam (kernel), the area of overlap (convolution output at each position). This “one metaphor, fully exploited” structure means the reader never has to switch mental models mid-slide — every new term is anchored to a scene they already understand. In contrast, `math.pdf` and `visual.pdf` are technically accurate but assume prior familiarity with the Toeplitz matrix representation and signal-processing vocabulary, creating a steep entry point for students who encounter convolution for the first time. `code.pdf` is highly practical but requires Python fluency to extract the conceptual insight, making it a complement rather than a foundation. The detective analogy succeeds because it encodes the **directional sliding** of the kernel and the **local multiplication-then-summation** operation in a scenario that is intuitively familiar, without requiring the reader to hold a formula in working memory.

In my own `v4_final.tex`, I emulated a similar “analogy-first” philosophy for Pipelining Hazards: the restaurant kitchen (parallel stages), the relay race (RAW dependence), the traffic officer (stall vs. forwarding), the loading dock (structural conflict), and the GPS recalculation (control hazard) each anchor one distinct concept before its formal definition appears. However, I made a different structural choice: I used **six separate analogies** — one per concept — rather than one sustained metaphor across the entire deck. This trades narrative coherence for coverage breadth. The instructor’s single-metaphor approach is arguably superior for initial conceptual loading, because a student who leaves mid-lecture still retains the full detective story and can reconstruct the concept later. My multi-analogy approach risks fragmentation: if a student forgets the relay-race frame, the RAW definition stands alone without scaffolding. A future iteration of my slides could consolidate around a **single assembly-line factory** metaphor — workers, conveyor belts, blocked stations, and rerouted parts — that could sustain all three hazard types without requiring a new mental scene per slide.

2. Information Density (TikZ Prompt)

The densest slide in `v4_final.tex` is **Slide 10 — “Control Hazards”**. It contains: a 3-line GPS analogy in italics, five bullet points (branch flush cost, static prediction, dynamic 2-bit counter, Branch Target Buffer, delayed branch), and an Intuition block — all without a single diagram. The critical missing visual is a **2-bit saturating counter state machine**: without it, the phrase “2-bit counter indexed by the lower bits of the PC” is memorisable but not understandable. A student cannot verify or apply the prediction logic without knowing the four states, the transition conditions, and which states produce which prediction.

The following prompt would generate this missing diagram:

Generate a TikZ diagram of a 2-bit saturating branch prediction state machine.

Nodes (4 states, arranged left to right):

- "Strongly\nNot Taken (00)" [fill=red!25]
- "Weakly\nNot Taken (01)" [fill=red!10]
- "Weakly\nTaken (10)" [fill=green!15]
- "Strongly\nTaken (11)" [fill=green!30]

Each node: draw, circle, minimum size=1.5cm, font=\scriptsize\bfseries.

Use \usetikzlibrary{automata, positioning, arrows.meta}.

Transitions:

- Left-to-right arcs (above nodes), labeled "Taken",
using -{Stealth}, bend left=20
- Right-to-left arcs (below nodes), labeled "Not Taken",
using -{Stealth}, bend left=20
- Self-loop at node 00 for "Not Taken"; self-loop at node 11 for "Taken"

Add a horizontal dashed line between states 01 and 10 labeled
"Prediction boundary" in gray.

Add a legend (two small colored rectangles): red = "Predict Not Taken",
green = "Predict Taken".

Scale the entire picture to fit inside a 10cm wide beamer column.

Output valid LaTeX inside \begin{tikzpicture}...\end{tikzpicture}.

Why this resolves the gap: The text bullet “dynamic prediction (2-bit counter): the CPU remembers past behaviour” conveys *that* the counter exists but not *how* it transitions. A student reading the slide cannot answer: “Does a single taken branch flip the prediction from Strongly Not Taken to Taken?” (Answer: no — it takes two consecutive taken branches to cross the prediction boundary.) The state-machine diagram makes the hysteresis property immediately visible: the boundary between red and green states shows that the predictor is inertially resistant to a single outlier branch, which is exactly why 2-bit counters outperform 1-bit counters on loop exit branches.

3. Efficiency

The part of my slides where AI-generated explanation clearly outperforms a standard textbook is the **forwarding worked example on Slide 8**. Patterson & Hennessy’s *Computer Organization and Design* (5th ed., Chapter 4) presents forwarding through a complete datapath diagram: multiplexer inputs, pipeline register labels, hazard detection unit signals, and forwarding unit control lines — spread across multiple figures spanning several pages. A student must mentally trace four separate figures to understand the single insight “the EX/MEM register’s output can bypass the register-file write-back.” The AI-generated version collapses this into one 13cm-wide slide: two colour-coded timing grids (blue cells for active stages, red cells for stalls, green dashed arrow for the forwarded value), a three-row \begin{tabular} showing the cycle-by-cycle position of each instruction, and a one-line quantitative conclusion.

The specific excerpt that demonstrates the efficiency gain:

“Without forwarding: 2 stalls (8 cycles). With forwarding: 0 stalls (6 cycles) — 25% speedup.”

A textbook gives the same numerical result but buries it in a paragraph following three figures. The AI version makes it the *punchline* of the slide — the single sentence a reader will remember and can immediately verify against the timing grid above it. This is a case where the AI’s output format (tightly constrained by the prompt) enforces pedagogical clarity that a textbook, constrained by its own conventions, tends to resist. The AI also generates **two complementary representations** of the same fact simultaneously (the visual timing grid and the symbolic table), which aligns with dual-coding theory: information encoded in both visual and verbal form is retained more reliably than information encoded in either form alone.

4. Slide Improvement Challenge

The most effective instructor slide is **c-real-life.pdf** (Convolution, detective analogy). Its weakness is that the analogy-to-formula mapping happens implicitly: the reader is expected to perform the translation themselves (“oh, the overlap area *is* the integral”). A stronger version would make the mapping **explicit and progressive** — showing the analogy and the formula side-by-side, building one term at a time.

The prompting technique used below is **Chain-of-Thought + Role-prompting + Constraint-prompting**.

You are a professor who has taught signal processing for 15 years and specialises in bridging intuition and formalism for students with no prior DSP background.

Your task: improve the "c-real-life" Convolution slide that currently uses a detective-with-flashlight analogy. The analogy is effective but the mapping from story to formula is implicit. Redesign it as a 3-frame progressive-disclosure sequence using `\pause` or `\onslide` in Beamer:

Frame 1 - Analogy only (no equations):

Describe the detective scanning a dark room with a flashlight.
Label the elements: flashlight beam = kernel $h()$, scene = signal $x(t)$,
overlap brightness = output $y(t)$.
Use a TikZ picture: a horizontal line (timeline), a sliding shaded rectangle
(kernel window), and a dot moving right (output sample being computed).

Frame 2 - Bridge (analogy + formula side by side, two columns):

Left column: the same TikZ diagram, now annotated with variable names.
Right column: the convolution integral $y(t) = \int x(\tau) h(t-\tau) d\tau$,
with each symbol colour-coded to match the diagram annotation.
Reveal each symbol with `\onslide<2->` so the formula builds one term at a time.

Frame 3 - Worked numerical example:

Discrete case: $x = [1, 2, 3]$, $h = [0, 1, 0.5]$.
Show the sliding-window multiplication in a tabular with `\cellcolor` for
the active kernel window.

Conclude: " $y[n] = \sum x[k] h[n-k]$ " - the formula is now just a compact notation for what you already watched happen visually.

Constraints:

- Use `\usetheme{metropolis}`, font size `\small` inside frames.
- Each frame must fit on one beamer slide (max 8 lines of text + diagram).
- No new mathematical prerequisites beyond multiplication and summation.
- Output valid LaTeX Beamer code only; do not explain your choices.

Why this improvement matters: The original `c-real-life.pdf` succeeds at motivation but leaves the formula introduction as an abrupt gear-shift. The refined version eliminates that discontinuity by using progressive disclosure — the formula is built symbol-by-symbol while the diagram is still on screen, so students are never asked to hold two separate representations in working memory simultaneously.

5. Missing Piece

The critical missing element in **my own slides** (`v4_final.tex`) is **failure-mode analysis**: what happens when the mitigation strategies break down.

The slides demonstrate the “happy path” thoroughly: forwarding eliminates 2 stalls in a clean 2-instruction RAW case; dynamic prediction wins >95% of the time; compiler scheduling eliminates known hazards at compile time. But they do not address:

- **The load-use case is stated but never shown visually.** Slide 7 mentions “one stall is unavoidable” for a load followed by a dependent instruction, but there is no timing diagram for it. A student who only read Slides 7 and 8 might incorrectly conclude that forwarding always produces zero stalls (the “Forwarding solves all data hazards” misconception identified in Slide 14’s pitfalls section was introduced without the supporting visual evidence needed to convince a skeptical reader).
- **No superscalar / out-of-order failure case.** The Mitigation Summary and Real-World Context slides mention that modern x86 CPUs use Tomasulo’s algorithm to handle RAW dynamically, but they do not explain what this costs: register rename table size, reorder buffer space, and the fact that Tomasulo does not solve memory-order hazards (store-to-load forwarding violations that require pipeline flushes). A student reading Slide 13 might conclude that out-of-order execution “solves” hazards — which is precisely the kind of overconfident partial understanding that produces bugs in systems programming.

How this gap affects understanding: The absence of failure-mode analysis creates a deceptively optimistic picture of pipeline design. Hazard mitigation is presented as a solved problem, when in practice it is a trade-off surface: forwarding adds wires and gate delay; deeper branch predictors consume die area and power; out-of-order windows have hard size limits that break down under irregular memory access patterns. A student who leaves with only the “success story” narrative will struggle to reason about why real CPUs stall visibly under specific workloads — and will have no framework for debugging the performance anomalies that appear in profiler output. A single additional slide titled “**When Mitigation Fails**” — showing the load-use timing diagram, a branch predictor aliasing scenario, and a note on structural out-of-order limits — would close this gap without significantly inflating the deck.

Part 4: Quality Assurance and Interactive Testing

a) Verification Quiz

Question 1 — Visual / TikZ Interpretation

In the “**With Forwarding**” TikZ timing diagram on the right column of the “**Worked Example: RAW Hazard & Forwarding**” slide (Slide 8 in `v4_final.tex` / Slide 6 in `v3.tex`), a green dashed arrow is drawn between two named nodes.

- (a) What are the LaTeX node names of the arrow’s source and target, as defined in the `\begin{tikzpicture}` block? What pipeline-stage labels (IF / ID / EX / ME / WB) do those nodes carry?
- (b) Which clock cycles (C1–C6 as labelled in the diagram’s header row) do the source node and the target node occupy?
- (c) The source node sits at (1.50, 0.45) and the target node sits at (2.10, 0.00) in the TikZ coordinate system. Why is the target node shifted **0.60 cm to the right** and **0.45 cm below** the source node? What does that spatial relationship encode about the instruction sequence?
- (d) The arrow is drawn with `\draw[fwdarr] (i1ex.south) to[out=-80, in=80]`. What would happen to the pipeline execution of I2 if this arrow — and its underlying hardware path — did not exist?

Expected Answer:

- (a) The source node is named `i1ex` and carries the label **EX** (I1’s execute stage). The target node is named `i2ex` and also carries the label **EX** (I2’s execute stage). Both are defined with the `blk` TikZ style (blue-filled, 0.60 × 0.38 cm rectangle).
- (b) `i1ex` is placed at $x = 1.50$, which corresponds to the **C3** column in the header row (cycle-number nodes are spaced 0.60 cm apart starting at $x = 0.30$ for C1). `i2ex` is placed at $x = 2.10$, corresponding to **C4**. The diagram therefore shows I1 executing in clock cycle 3 and I2 executing in clock cycle 4 — back-to-back with no gap.
- (c) The 0.60 cm rightward offset reflects I2 being issued **one cycle later** than I1 (each cycle column is 0.60 cm wide). The 0.45 cm downward offset places the two instructions on **separate rows** — I1 at $y = 0.45$ (the upper instruction row) and I2 at $y = 0.00$ (the lower instruction row). Together, the spatial relationship encodes the timing-diagram convention: column = clock cycle, row = instruction, so the diagonal step-down from `i1ex` to `i2ex` visually captures the fact that two consecutive instructions reach their EX stages in two consecutive clock cycles.
- (d) Without the forwarding path, I2 would need to read R1 from the **register file** at the start of its EX stage. R1 is not written to the register file until I1 completes its **WB** stage at the end of cycle C5. Therefore, I2’s EX stage could not begin until cycle C6. The hazard detection unit would freeze I2 in the ID stage for **two stall cycles** (C4 and C5), inserting two NOP bubbles into the

pipeline. The timing diagram would look like the left-column “Without Forwarding” grid: eight total cycles instead of six, with two red -- cells visible at (I2, C4) and (I2, C5).

Question 2 — Logical Derivation / Calculation

Consider the following **3-instruction RAW chain** in a 5-stage pipeline (IF → ID → EX → MEM → WB):

```
I1:  ADD  R1, R2, R3      ← writes R1
I2:  SUB  R4, R1, R5      ← reads R1  (RAW on R1 from I1)
I3:  AND  R6, R4, R7      ← reads R4  (RAW on R4 from I2)
```

- Build the complete cycle-by-cycle stage table **without forwarding**, marking every stall slot as --. Show all cycles until I3's WB stage completes.
- Count the total number of stall cycles and identify which instruction pair causes each group of stalls.
- Repeat (a) **with full forwarding** (EX→EX and MEM→EX bypass paths active).
- Calculate the total cycle count in both cases and the absolute speedup (ratio of cycles without forwarding to cycles with forwarding).

Expected Answer:

(a) **Without forwarding — cycle-by-cycle table:**

	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11
I1	IF	ID	EX	ME	WB						
I2		IF	ID	--	--	EX	ME	WB			
I3			IF	IF	IF	ID	--	--	EX	ME	WB

Explanation of each stall group:

- I2 at C4–C5:** I2 enters ID at C3 and would normally enter EX at C4. Without forwarding, it needs R1 from the register file, but I1 completes WB at the *end* of C5. The hazard detection unit holds I2 in ID and freezes I3 in IF for two cycles. I2 can enter EX at C6, one cycle after R1 is committed to the register file.
- I3 at C7–C8:** I3 enters ID at C6 (after being held in IF for three cycles by I2's stalls) and would normally enter EX at C7. Without forwarding, it needs R4 from the register file, but I2 completes WB at the *end* of C8. I3 is held in ID for two more cycles. I3 can enter EX at C9.

(b) **Total stall count:**

Instruction pair	Stall cycles	Cycles
I1 → I2 (RAW on R1)	2 stalls	C4, C5

Instruction pair	Stall cycles	Cycles
I2 → I3 (RAW on R4)	2 stalls	C7, C8
Total	4 stalls	

(c) With full forwarding — cycle-by-cycle table:

	C1	C2	C3	C4	C5	C6	C7
I1	IF	ID	EX	ME	WB		
I2		IF	ID	EX	ME	WB	
I3			IF	ID	EX	ME	WB

- **I1 → I2:** R1 exits I1's EX stage at the end of C3. The EX→EX forwarding path routes it directly to I2's ALU input at the start of C4. Zero stalls.
- **I2 → I3:** R4 exits I2's EX stage at the end of C4. The EX→EX forwarding path routes it to I3's ALU input at the start of C5. Zero stalls.

(d) Cycle counts and speedup:

	Without forwarding	With forwarding
Total cycles	11	7
Stall cycles	4	0
Ideal ($n + k - 1$)	$3 + 4 = 7$	$3 + 4 = 7$

$$\text{Speedup} = \frac{11}{7} \approx 1.571 \quad (57.1\% \text{ faster with forwarding})$$

Derivation from the slide formula: the slides state that for a 2-instruction RAW case, forwarding eliminates 2 stalls and delivers a 25% speedup (8 cycles → 6 cycles). Scaling to a 2-RAW chain doubles the stall savings: 4 stalls removed from 11 cycles yields 7 cycles, consistent with the formula $\text{total_cycles} = n + (k - 1) + \text{stall_count}$ where $n = 3$ instructions, $k = 5$ stages.

b) Decompression Task

Most complex slide: “**Worked Example: RAW Hazard & Forwarding**” (Slide 8 in `v4_final.tex` — contains two TikZ timing grids, a `\begin{tabular}`, a forwarding arrow, and quantitative conclusion text.)

You are a teaching assistant specialising in computer architecture for students who have never seen a pipeline timing diagram before.

Take the following Beamer slide content (from `v4_final.tex`, Slide 8):

```

---
\begin{frame}{Worked Example: RAW Hazard \& Forwarding}

\texttt{\scriptsize I1:~ADD~R1,R2,R3} \quad
\texttt{\scriptsize I2:~SUB~R4,\textcolor{red}{R1},R5}
\;{\scriptsize --- I2 reads R1 produced by I1 (\textbf{RAW} dependence)}

\begin{columns}[T]
  \begin{column}{0.50\textwidth}
    \centering{\small\textbf{Without Forwarding}}
    % [TikZ grid: 8 cycles, I1 stages IF-ID-EX-ME-WB, I2 with 2 red -- stalls
    % then EX-ME-WB]
    \centering{\scriptsize \textcolor{red}{2 stall cycles} 8 total cycles}
  \end{column}
  \begin{column}{0.50\textwidth}
    \centering{\small\textbf{With Forwarding}}
    % [TikZ grid: 6 cycles, I1 IF-ID-EX-ME-WB, I2 IF-ID-EX-ME-WB,
    % green dashed arrow from i1ex.south to i2ex.north labelled "fwd"]
    \centering{\scriptsize \textcolor{green!60!black}{0 stalls} 6 total cycles}
  \end{column}
\end{columns}

\begin{tabular}{l|cccccccc}
\hline
Instruction & C1 & C2 & C3 & C4 & C5 & C6 & C7 & C8 & \\
\hline
I1 (ADD R1,R2,R3) & IF & ID & EX & ME & WB & & & & \\
I2 without fwd & & & IF & ID & -- & -- & EX & ME & WB \\
I2 with fwd & & & IF & ID & EX & ME & WB & & \\
\hline
\end{tabular}

\textbf{Result:} No fwd: 2 stalls (8 cycles). Fwd: 0 stalls (6 cycles) --- 25\% speedup.
\end{frame}
---

```

Split this into a 3-frame sequence for a complete beginner who has never seen a pipeline timing diagram:

Frame 1 - Visual intuition only (no equations, no formula):

- Show ONLY the two-column TikZ timing grid (before/after).
- Add a plain-English caption under each grid:
 - "Without forwarding: I2 waits 2 cycles for R1 to be written."
 - "With forwarding: R1 is handed directly from I1's execute stage to I2's."
- Do NOT include the tabular or any formula.
- Add a `\begin{block}`{What you are seeing} block explaining what a row, a column, and a red -- cell mean, in one sentence each.

Frame 2 - Introduce the stall-count formula with progressive disclosure:

- Start with the sentence: "Total cycles = Ideal cycles + Stall cycles"
- Use `\onslide<2->` or `\pause` to reveal each term one at a time:
 - `\onslide<1->`: "Total cycles = ?"
 - `\onslide<2->`: "+ Ideal cycles: $n + (k-1) = 2 + 4 = 6$ "
 - `\onslide<3->`: "+ Stall cycles: 2 (one per cycle I2 is frozen in ID)"
 - `\onslide<4->`: "= Total: $6 + 2 = 8$ cycles"
- Add a `\begin{block}{Hint}` at the bottom:

"n = number of instructions, k = pipeline depth (5 stages). The formula counts cycles from the moment the first instruction enters IF until the last instruction completes WB."

Frame 3 - Walk through ONE concrete numerical example step by step:

- Use ONLY the "without forwarding" row of the tabular.
- Show the table one column at a time using `\onslide` or `\pause`:
 - Step 1: I1 enters IF at C1. I2 enters IF at C2.
 - Step 2: I1 is in EX at C3. I2 is in ID at C3. Does I2 have R1 yet? No.
 - Step 3: I1 is in MEM at C4. I2 is FROZEN in ID. Stall 1.
 - Step 4: I1 is in WB at C5. I2 is FROZEN in ID. Stall 2.
 - Step 5: I1 is done. I2 enters EX at C6. R1 is now in the register file.
 - Step 6: I2 finishes at WB in C8. Total: 8 cycles.
- After step 6, add `\pause` and reveal: "With forwarding: skip steps 3-4. Total: 6 cycles. Savings: 2 cycles = 25%."
- Add a `\begin{alertblock}{Hint}` at the bottom:

"The stall ends the cycle AFTER the producer completes WB - because the register file write happens at the end of the WB clock edge."

Constraints:

- Use `\usetheme{Madrid}`.
- Each frame max 6 lines of visible text (excluding block environments).
- The Hint / alertblock must appear at the bottom of frames 2 and 3.
- Output valid LaTeX Beamer code only; do not explain your approach.
- Do not use emoji in LaTeX source.

Expected output note:

Running this prompt against a capable language model would produce three self-contained `\begin{frame}...\end{frame}` blocks in the Madrid Beamer theme. Frame 1 would reproduce the two TikZ timing grids from the original slide but strip the tabular and formula, adding a `\begin{block}{What you are seeing}` with three one-sentence definitions. Frame 2 would build the formula $\text{Total} = \text{Ideal} + \text{Stalls} = (n + k - 1) + 2$ using `\onslide<1->` through `\onslide<4->` overlays, so the equation assembles itself one term at a time on successive clicks — ensuring students absorb each symbol before the next appears. Frame 3 would present the tabular row-reveal as a narrated walk-through, with each `\pause` corresponding to one clock cycle of the execution trace, and conclude with the forwarding comparison and an `\begin{alertblock}{Hint}` clarifying the off-by-one subtlety (the stall ends the cycle *after* WB, not during it). The most likely generation error would be misaligned `\onslide` counters in Frame 2 — a known weakness

of LLM-generated Beamer overlay code — requiring one manual check pass to ensure the overlay numbers are consecutive and non-overlapping.

Part 5: Feedback on AI Synthesis

1. Hardest Elements to Get the AI to Generate Correctly

TikZ Diagrams

The core difficulty is that TikZ requires **exact coordinate arithmetic** that the AI cannot verify without a compile step. For the timing grids in `v3.tex` and `v4_final.tex`, every cell position had to be computed manually ($x = 0.30 + 0.60 \times n$ for column n) — a prompt that says “draw a pipeline timing diagram” without specifying coordinates produces nodes that overlap, spill outside the slide boundary, or use inconsistent spacing. The forwarding arrow in the “With Forwarding” diagram required explicit angle parameters (`to[out=-80, in=80]`): without them, a generic curved-path instruction generates either a nearly-invisible tight curve or an overshooting arc that exits the frame. A third failure mode is **silent library omission**: the `{Stealth}` arrowhead in `harr/.style` and `fwdarr/.style` produces no arrowhead and no error if `\usetikzlibrary{arrows.meta}` is missing — making bugs invisible until the PDF is opened.

Mathematical Notation

The most persistent LaTeX issue was **colour-scoping inside tabular environments**. The `\textcolor{red}{--}` syntax correctly scopes colour to a single cell, but AI-generated code often substitutes `\color{red}{--}`, which leaks colour to every subsequent cell in the same row until the group closes — producing a red row instead of two red cells. A second source of errors was inline math symbols outside math mode: `,`, `\rightarrow`, and `\approx` pasted as Unicode characters fail silently or produce garbled output in `pdflatex`; the correct forms `\approx`, `\to`, and `\rightarrow` must be explicit. The 8-column `\begin{tabular}` in the worked example additionally required `\setlength{\tabcolsep}{3.5pt}` to prevent overflow — a micro-layout adjustment that no generic AI prompt anticipates, because the AI has no model of the slide’s physical width.

Pedagogical Balance

Calibrating the analogy-to-definition ratio in each iteration was difficult because the AI’s default is to **front-load the formal definition and append the analogy**, whereas effective pedagogy reverses that order (analogy first, definition second, as a label for something already understood). In `v1.tex`, bullets like “RAW (true dependence): a consumer instruction attempts to read a register operand before the producer instruction completes its WB stage” require prior familiarity with six technical terms before the concept becomes accessible. The Intuition blocks in `v4_final.tex` presented the opposite risk: condensing six slides’ worth of content into 1–2 sentences each, the AI’s tendency is to **paraphrase the bullet points** (“Forwarding allows results to bypass the register file”) rather than distil the underlying insight (“Forwarding trades extra wires for time — the load-use case is the one penalty no wiring can eliminate”).

2. Repeated AI Mistakes

Verbose explanations in bullet points. In `v1.tex`, every bullet was 2–3 lines long: “*RAW (true dependence): a consumer instruction attempts to read a register operand before the producer*”

instruction completes its WB stage.” A beamer bullet should fit on one line; this reads like a textbook paragraph. A human lecturer would write: “*RAW: instruction B reads a register before instruction A finishes writing it.*” The verbosity forced `\medskip` between items just to create visual breathing room, compressing usable slide area.

Missing concrete examples in the first draft. `v1.tex` generated five content slides without a single instruction pair (e.g., `ADD R1, R2, R3` followed by `SUB R4, R1, R5`). Every hazard was described at the abstract level — “a consumer instruction,” “a producer instruction” — with no instance that bound the concept to actual registers and values. The worked example (with `I1`, `I2`, and cycle-by-cycle tracking) only appeared in `v3.tex` after an explicit prompt demanding a `\begin{tabular}` numerical example.

Wrong abstraction level for the target audience. The Hazard Mitigation Summary in `v1.tex` used graduate-seminar language: “*The hazard detection unit asserts a stall signal, holding PC and IF/ID registers constant while inserting a bubble (NOP) into the ID/EX register.*” For an undergraduate audience encountering pipelining for the first time, this sentence assumes knowledge of PC registers, pipeline register names, and signal assertion — none of which were defined earlier in the slide deck. The audience mismatch persisted until `v2.tex` explicitly targeted a “10-year-old” comprehension level.

Speculative package loading in the LaTeX preamble. `v1.tex` loaded `\usepackage{listings}` without any `\lstset{}` configuration or `\begin{lstlisting}` environment anywhere in the document. The `listings` package was listed in the assignment requirements but served no purpose in `v1.tex`; AI-generated preambles routinely include packages pre-emptively “in case they are needed,” creating unnecessary compilation overhead and potential conflicts (the `listings` First Aid warning visible in the `v3.tex` compile log was a direct consequence of this pattern).

Structurally repetitive Intuition blocks. Three of the six `\begin{block}{Intuition}` blocks in `v4_final.tex` follow the same two-clause sentence structure: “*X is the only Y; Z vanishes when W.*” / “*X adds Y; Z is the one penalty no Y can eliminate.*” / “*X patches runtime hazards; Y eliminates compile-time ones.*” While structural parallelism aids memory, the near-identical sentence frames make the blocks feel formulaic rather than tailored to each concept’s specific insight. The Structural Hazards block (“Every structural hazard has one root cause: more demand than supply”) avoids this pattern and is noticeably sharper for it.

3. Golden Rule Prompt

You are an expert computer science tutor who has taught [TOPIC] to undergraduate students for 10 years. Explain [TOPIC] in three escalating layers:

1. Intuition - no jargon: Describe the concept using a single real-world analogy in 2-3 sentences of prose. Do not use any technical term a 12-year-old would not already know. Do not use bullet points in this layer.
2. Formal definition: State the precise definition, introducing each new technical term in parentheses the first time it appears. Max 4 bullet points.
3. Worked example: Walk through ONE concrete numerical example step by step, showing every intermediate value explicitly. Conclude with a one-sentence rule of thumb that a student can apply in an exam without re-reading the definition.

Constraints:

- Total response: max 300 words across all three layers.
- Layer 3 must include at least one numerical value or formula substitution.
- Difficulty level: [BEGINNER / INTERMEDIATE / ADVANCED] - calibrate vocabulary, example complexity, and assumed prior knowledge accordingly.
- After Layer 3, generate exactly 2 self-test questions at the specified difficulty level. Do NOT provide the answers - the reader must attempt them independently.

This prompt is the “golden rule” because it encodes two well-validated pedagogical principles simultaneously. The three-layer structure mirrors **Bruner’s spiral curriculum**: intuition first (enactive representation), formal definition second (symbolic representation), and a worked example third (which forces the student to verify that the symbol system predicts the concrete outcome) — ensuring that no layer is learned in isolation from the others. The “no answers” directive on the self-test questions enforces **active retrieval** rather than passive re-reading: a student who must generate an answer, even incorrectly, retains the material significantly longer than one who reads a provided answer, a finding replicated across decades of cognitive science research under the label of the testing effect. The difficulty parameter makes the prompt reusable across all stages of exam preparation — running it at BEGINNER level two weeks before an exam and at ADVANCED level the night before produces two complementary study sessions from a single template.